MINISTRY OF EDUCATION AND SCIENCE OF THE REPUBLIC OF TAJIKISTAN
DUSHANBE INNOVATION INSTITUTE

Faculty “Exact and mathematical sciences”
Department “Exact and mathematical sciences”

COURSEWORK

in Algorithms and Data Structures

on the topic: “Lossless Text File Compression Using Huffman Coding”

Prepared by:
Student of 2nd year
Group Computer Science
Islomzhon Ibragimov

Supervisor: Associate Professor, Kholiqov O.

Dushanbe - 2026

TABLE OF CONTENTS

Introduction

File compression is an important topic in computer science because it helps store and transfer information using fewer bytes. Text files, logs, source code files, and messages often contain repeated characters and repeated patterns. If these repeated parts are represented in a shorter form, the size of the file can be reduced. This is useful when disk space is limited or when data needs to be sent through a network.

The topic of this coursework is lossless text file compression using Huffman coding. Lossless compression means that the original information can be restored exactly after decompression. This is different from lossy compression, where some information is removed to save more space. For text files, lossless compression is necessary because even one wrong character can change the meaning of the file.

Huffman coding is a classical algorithm for lossless compression. It was introduced by David Huffman in 1952 and is still used as a basic idea in many compression systems. The main idea is simple: characters that appear more often should have shorter binary codes, and characters that appear rarely should have longer binary codes. In this way, the average number of bits per character becomes smaller.

This project is connected with Algorithms and Data Structures because it uses several basic structures together. A dictionary is used to count character frequencies. A priority queue is used to build the Huffman tree. A binary tree is used to store the code structure. The program also uses traversal of the tree to generate codes and to decode compressed data.

The aim of the project is to design and implement a small working program that can compress and decompress text files using Huffman coding. The program should show that the algorithm works correctly by decompressing the compressed file back to the same original text.

Aim of the project

The main aim is to implement a Huffman file compression tool and explain the algorithms and data structures used in it.

Objectives

The objectives of the project are:

1. Read a text file from the computer.
2. Count how many times each character appears.
3. Build a Huffman tree using a min heap.
4. Generate a binary code for every character.

5. Compress the text into a binary file.
6. Decompress the binary file back into text.
7. Test the program with different examples.
8. Analyze time and space complexity.

Subject and scope

The subject of the project is the Huffman coding algorithm and its practical implementation for text files. The scope is limited to text file compression and decompression. The program does not try to compete with professional compression tools. It is created for learning and demonstrating how Huffman coding works internally.

The project uses Python because it is easy to read and suitable for algorithm implementation. Python also has a standard heap library, which helps implement the priority queue without writing a heap from zero. The algorithm itself is still implemented directly: the program builds its own tree, generates its own codes, stores data in its own simple format, and decodes the file again.

This work is not focused on image, audio, or video compression. These files use more complex methods and often combine many algorithms. The project also does not use real bit-level compression libraries. The purpose is to show the algorithm clearly and test it with simple text files.

Importance of the topic

Compression is important because digital data grows every year. People store documents, programs, databases, and communication records. If data can be represented in a smaller form, it saves storage and sometimes reduces transfer time. Even when storage is cheap, compression is still useful in backups, archives, messaging, and network systems.

Huffman coding is also important from an educational point of view. It connects theory with practice very well. The student can see why a greedy algorithm works, how a tree can represent codes, and why a priority queue is useful. The final program is also easy to demonstrate because the user can compress a file and then decompress it.

For this coursework, Huffman coding is a good choice because it includes enough algorithmic content, but the program is still possible to implement in a clear way. The project is not only theoretical. It produces a working file and checks whether the decompressed result is exactly the same as the input.

Limitations

The main limitation is that very small files may become larger after compression. This happens because the compressed file must also store information needed for decompression, such as

the frequency table. For a small input, this extra header can be bigger than the saving from Huffman codes. This does not mean the algorithm is wrong. It shows that compression has overhead.

Another limitation is that the program is mainly designed for text. It reads input using UTF-8 encoding and writes output as text. Binary files are not the main goal of this coursework version. Also, the file format is simple and made for this project, not for professional use.

Chapter 1. Theoretical Background

1.1 Data compression

Data compression means representing information with fewer bits than the original representation. There are two main types of compression: lossless and lossy. Lossless compression keeps all information exactly. After decompression, the result is identical to the original. Lossy compression removes some information to make files smaller. It is often used for images, audio, and video.

Text compression usually needs to be lossless. If a text document is compressed and later restored, every letter, space, punctuation mark, and line break should be the same. In programming files, even one missing symbol can make the program incorrect. Because of this, a Huffman coding tool for text must decompress the file exactly.

Compression is possible when the data has redundancy. Redundancy means some symbols or patterns appear more often than others. For example, in English text, spaces and letters such as e, t, and a may appear many times. Rare letters or symbols appear less often. Huffman coding uses this difference in frequency.

A fixed length code uses the same number of bits for every symbol. For example, normal character encodings often use one byte or more for each character. Huffman coding uses variable length codes. A frequent character may use only two or three bits, while a rare character may use seven or eight bits. If the frequent characters really appear often, the total size becomes smaller.

1.2 Prefix codes

A prefix code is a code where no code word is the prefix of another code word. This property is very important for decoding. If no code is a prefix of another code, the decoder can read bits from left to right and know exactly when one character code ends.

For example, suppose the code for a is 0, the code for b is 10, and the code for c is 11. These codes are prefix-free. If the decoder reads 0110, it can split it as 0 11 0, which means a c a. There is no confusion because 0 does not start any longer code and 10 and 11 are separate paths.

If codes are not prefix-free, decoding may become ambiguous. For example, if a is 0 and b is 01, then the bit sequence 01 can mean b or it can start with a followed by something else. Huffman coding avoids this problem by using a binary tree.

In a Huffman tree, each character is stored in a leaf node. Moving left adds 0 to the code, and moving right adds 1. Since characters are only stored in leaves, no character code can be the prefix of another character code. This is why the Huffman tree automatically gives prefix codes.

1.3 Huffman coding

Huffman coding is a greedy algorithm. A greedy algorithm makes the best local choice at each step. In Huffman coding, the best local choice is to combine the two symbols or subtrees with the smallest frequencies. The algorithm repeats this until only one tree remains.

The steps of Huffman coding are:

1. Count the frequency of every character.
2. Create a leaf node for each character.
3. Insert all nodes into a min heap.
4. Remove two nodes with the smallest frequencies.
5. Create a new parent node with their combined frequency.
6. Insert the new node back into the heap.
7. Repeat until one node remains.
8. Traverse the final tree to create binary codes.

The final tree is called the Huffman tree. More frequent characters usually appear closer to the root, so their codes are shorter. Less frequent characters appear deeper in the tree, so their codes are longer. The algorithm creates an optimal prefix code for the given set of frequencies.

For example, if a file contains many spaces and only a few z characters, the space character should receive a shorter code than z. This does not mean every individual character gets shorter. Some rare characters may get longer codes. The benefit comes from the total number of bits used for the whole file.

The Huffman algorithm is suitable for this project because the input is known before compression. The program reads the whole text, counts frequencies, and builds the tree from that exact input. This is called static Huffman coding because the tree is built once from the input data.

1.4 Binary trees

A binary tree is a data structure where each node can have at most two children. The children are usually called left and right. In this project, the Huffman tree is a binary tree. Internal nodes do not store characters. They only store combined frequency values. Leaf nodes store actual characters.

The path from the root to a leaf gives the code of a character. A left movement represents bit 0, and a right movement represents bit 1. If the path to character x is left, right, left, then the code for x is 010.

Tree traversal is used for code generation. Starting from the root, the program recursively visits the left and right child. The code string grows during the traversal. When a leaf is found, the current code string is stored in a dictionary.

The same tree is also used during decompression. The decoder starts at the root and reads one bit at a time. If the bit is 0, it goes left. If the bit is 1, it goes right. When it reaches a leaf, it writes the character and returns to the root for the next character.

The tree structure is simple but powerful. It stores all variable length codes in a way that makes decoding direct. There is no need to search all codes for every character while decoding. The path through the tree decides the character.

1.5 Priority queues and heaps

A priority queue is a data structure where each item has a priority. The item with the smallest or largest priority can be removed efficiently. In Huffman coding, the priority is the frequency of a node. The algorithm always needs the two nodes with the smallest frequencies.

A min heap is a common implementation of a priority queue. In a min heap, the smallest item is kept at the top. Removing the smallest item and inserting a new item both take logarithmic time. This is much faster than sorting the full list after every merge.

If there are k unique characters, the heap starts with k nodes. The algorithm removes two nodes and inserts one merged node each time. This happens k - 1 times. Because heap operations are O(log k), building the Huffman tree takes O(k log k) time.

Python has a standard module named heapq that implements a min heap. In the project, heapq is used for priority queue operations. Each heap item stores the frequency, a small number to avoid comparison problems, and the tree node itself.

1.6 Dictionaries and frequency tables

A dictionary is used to store key-value pairs. In this project, the first dictionary maps characters to their frequencies. For example, the key can be the character a, and the value can be 25 if a appears 25 times. This makes counting simple because each character can be updated in constant average time.

A second dictionary is used for Huffman codes. After the tree is built, each character receives a binary code. The dictionary maps the character to this code. During compression, the program reads every character from the original text and replaces it with the code from the dictionary.

The frequency table is also stored inside the compressed file header. This is needed because the decompression program must rebuild the same Huffman tree. Without the frequency table or an equivalent tree description, the compressed bits cannot be decoded.

Using a dictionary is suitable because the program often needs fast lookup. When counting frequencies, it checks and updates characters many times. When encoding text, it looks up the code for each character. A dictionary gives good practical performance for both tasks.

Summary of chapter 1

The theory of the project is based on compression, prefix codes, Huffman coding, binary trees, priority queues, and dictionaries. These topics are connected in one complete algorithm. The dictionary counts symbols, the heap chooses the smallest frequencies, the binary tree stores the code structure, and the prefix property makes decompression possible.

Chapter 2. Analysis and Design

2.1 Problem statement

The problem is to create a program that takes a text file and produces a compressed file. The compressed file should be smaller for suitable input and must contain enough information to restore the original text. The program must also read the compressed file and create an output text file that is exactly the same as the original.

The main challenge is that variable length codes must be decoded correctly. If codes are chosen without structure, the decoder may not know where one code ends and another begins. Huffman coding solves this by creating prefix-free codes from a binary tree.

Another practical problem is file storage. The program cannot store just the encoded bits. It also has to store metadata. In this project, metadata means the frequency table and the number of padding bits added at the end. These values are stored in a simple header.

2.2 Input and output

The compression command receives two file paths. The first path is the source text file.

The second path is the destination compressed file. The program reads the source using UTF-8 encoding and writes the compressed result as binary data.

The decompression command also receives two file paths. The first path is the compressed file. The second path is the destination text file. The program reads the binary file, rebuilds the tree, decodes the bits, and writes the restored text using UTF-8 encoding.

The command format is:

```
python main.py compress input.txt output.huff
python main.py decompress input.huff output.txt
```

The program prints simple information after running. During compression, it prints the original size, compressed size, number of unique characters, compression ratio, and saved space. During decompression, it prints the number of output characters and the output size.

2.3 Chosen data structures

The project uses a Node class for the Huffman tree. Each node stores a character, a frequency, and links to the left and right children. Leaf nodes store characters. Internal nodes store no character and are used only to connect two smaller trees.

The frequency table is implemented with a dictionary. The program loops through every character in the input text and increases its count. This is a direct and simple method.

The priority queue is implemented with a min heap. Each heap entry contains the frequency, a number used only to keep order stable, and the node. The heap makes it efficient to repeatedly choose the two smallest nodes.

The code table is another dictionary. It maps each character to a string of 0 and 1 values. This representation is simple to understand and good enough for a coursework project. After all codes are created, the bit string is packed into bytes before writing the file.

2.4 Main algorithm

The compression algorithm starts by reading the full text. It counts frequencies and builds the Huffman tree. Then it walks through the tree to generate codes. After that, it replaces each character in the text with its Huffman code.

The resulting sequence of bits may not be divisible by 8. Since files are written in bytes, the program adds zero bits at the end. The number of added bits is called padding. This number is saved in the header so the decompression program can remove the extra bits.

The decompression algorithm reads the header first. From the saved frequency table, it rebuilds the Huffman tree. Then it reads the compressed bytes and converts them back to a string of bits. The saved padding value is used to remove the extra zeros. Finally, the program walks through the tree and restores the original characters.

The design is simple because the same frequency table creates the same Huffman tree. The compressed file does not need to store the whole tree directly. It stores the data needed to build the tree again.

2.5 File format

The compressed file uses a simple custom format:

First 4 bytes: size of the header Header: JSON data with frequency table and padding Remaining bytes: compressed data The first four bytes store the header size as an integer. This allows the decompression program to know exactly how many bytes belong to the header. After reading the header, the rest of the file is compressed data.

The header is written as JSON because it is easy to create and read in Python. It contains two fields: freq and pad. The freq field stores character frequencies. The pad field stores how many zero bits were added at the end.

This format is not the most compact possible because JSON has extra characters. However, it is easy to understand and explain. For a coursework project, clarity is more important than

maximum compression ratio. A more advanced version could store the tree or frequency table in a smaller binary form.

2.6 Complexity analysis

Let n be the number of characters in the input text. Let k be the number of unique characters. Usually, k is much smaller than n for normal text.

Counting character frequencies visits every character once, so its time complexity is O(n). The dictionary uses O(k) space because it stores one entry for each unique character.

Building the Huffman tree uses a heap with k elements. Removing two minimum elements and inserting one new node costs O(log k). This is repeated k - 1 times, so the time complexity is O(k log k). The tree uses O(k) space.

Generating codes visits each tree node. A Huffman tree with k leaf nodes has about 2k - 1 nodes, so code generation is O(k). The code table also uses O(k) entries, but the total size of code strings depends on the depth of the tree.

Encoding the text visits every character and looks up its code. This takes O(n) time. The final encoded bit string depends on the compression result, but in memory it may still be proportional to the input size.

Decoding visits each encoded bit and walks through the tree. The total number of bits depends on the generated codes. In simple analysis, decoding is written as O(n) in relation to the number of original characters, because each decoded character is produced once.

Complexity table

Operation	Time Complexity	Space Complexity
Count frequencies	O(n)	O(k)
Build Huffman tree	O(k log k)	O(k)
Generate codes	O(k)	O(k)
Encode text	O(n)	O(n)
Decode text	O(n)	O(n)

The total compression time is mainly O(n + k log k). Since k is usually smaller than n for text, the program is efficient for normal coursework examples.

2.7 Alternative approaches

One alternative is fixed length coding. It is simple, but it does not use character frequencies. Every character receives the same number of bits, so frequent characters do not get shorter codes. This can waste space.

Another alternative is run length encoding. It stores repeated runs such as aaaaa as a5. This works well when the same symbol appears many times in a row. However, it does not work well for normal text where repeated characters are not always consecutive.

Lempel-Ziv based methods are used in many real compression formats. They find repeated strings instead of only repeated characters. These methods can compress many files better than simple Huffman coding, but they are more complex. Huffman coding is more suitable for this coursework because the tree, heap, and frequency table are clear ADS topics.

Adaptive Huffman coding is another version where the tree changes while reading the input. It can work without storing the whole frequency table first. However, it is harder to implement correctly. This project uses static Huffman coding because it is simpler and easier to test.

Chapter 3. Practical Implementation

3.1 Programming language and files

The project is implemented in Python. Python was chosen because it is readable and suitable for studying algorithms. The program uses only standard libraries: heapq for the heap, json for storing the header, os for file size, and sys for command-line arguments.

The project has a small file structure:

```
main.py
huffman.py
README.md
samples/input.txt
samples/repeated.txt
```

The file main.py handles command-line arguments. It checks whether the user wants to compress or decompress. It calls functions from huffman.py and prints results.

The file huffman.py contains the main algorithm. It has functions for counting frequencies, building the tree, generating codes, packing bits into bytes, reading bits back, compressing, and decompressing.

The samples folder contains input files used during testing. The file input.txt contains a short normal paragraph. The file repeated.txt contains repeated characters and repeated words, which gives a better example of compression.

3.2 Important functions

The Node class represents a tree node. It stores a character, a frequency, and left and right children. If the character value is empty, the node is an internal node. If the character exists, the node is a leaf.

```
class Node:
def __init__(self, ch, freq, left=None, right=None):
self.ch = ch
self.freq = freq
self.left = left
self.right = right
```

The make_freq function counts how many times each character appears. It uses a dictionary and loops through the input text once.

```
def make_freq(text):
freq = {}
for ch in text:
freq[ch] = freq.get(ch, 0) + 1
return freq
```

The make_tree function builds the Huffman tree. It creates a heap from the frequency table. Then it repeatedly removes the two smallest nodes and joins them into a new parent node.

```
while len(heap) > 1:
f1, n1, left = heapq.heappop(heap)
f2, n2, right = heapq.heappop(heap)
root = Node(None, f1 + f2, left, right)
heapq.heappush(heap, (root.freq, num, root))
```

The make_codes function traverses the tree. Going left adds 0 to the code, and going right adds 1. When a leaf is found, the code is stored in the dictionary.

The make_bytes function packs a string of bits into bytes. Files are written as bytes, so a bit string such as 10101010 can be converted into one byte. If the last group has fewer than 8 bits, zeros are added and the number of added zeros is saved.

The make_bits function does the opposite. It reads bytes from the compressed file and converts every byte into eight bits. Then it removes the padding bits using the value stored in the header.

The read_bits function decodes the bit string using the Huffman tree. It starts from the root. For each bit, it moves left or right. When it reaches a leaf, it adds the character to the output and starts again from the root.

3.3 Program usage

The compression command is:

```
python main.py compress samples/input.txt samples/output.huff
```

Example output:

```
Compressed successfully
Original size: 245 bytes
Compressed size: 374 bytes
Unique characters: 26
Compression ratio: 1.527
Space saved: -52.65 %
```

This result shows that the first small sample became larger. The reason is that the compressed file also stores the frequency table. For a small file, this overhead is large compared to the text size.

The decompression command is:

```
python main.py decompress samples/output.huff samples/result.txt
```

Example output:

```
Decompressed successfully
Output characters: 245
Output size: 245 bytes
```

To check correctness, the diff command can be used:

```
diff samples/input.txt samples/result.txt
```

If diff prints nothing, the two files are the same. This was the result during testing.

3.4 Test cases

Several test cases were used to check the program. The most important condition is that decompression must restore the exact original file.

```
Test 1: Normal short text
Input file: samples/input.txt
Original size: 245 bytes
Compressed size: 374 bytes
Result: decompressed file matched original
```

```
Test 2: Repeated text
Input file: samples/repeated.txt
Original size: 280 bytes
Compressed size: 231 bytes
Space saved: 17.5 %
Result: decompressed file matched original
```

```
Test 3: Empty text file
Expected result: program creates a valid compressed file
and decompresses it back to an empty file.
```

```
Test 4: One repeated character
Expected result: the program handles the special case where
the Huffman tree has only one leaf.
```

3.5 Results

The program successfully compressed and decompressed the tested files. The decompressed output matched the original input in the tests that were run. This proves that the program is lossless for these cases.

The repeated sample showed real compression. It had many repeated characters and words, so Huffman coding gave shorter codes to the frequent symbols. The compressed file was smaller than the original.

The normal small sample became larger. This is an expected result for small files because the file header is included. If the file is larger or has many repeated symbols, the compression result improves.

The program is easy to demonstrate. A user can open a terminal, run the compression command, run the decompression command, and compare the output with the original file. The printed statistics also help explain what happened.

3.6 Limitations and possible improvements

The main limitation is that the header is stored as JSON. JSON is easy to read and simple to use, but it is not compact. A better version could store the frequency table in a binary format to reduce overhead.

Another limitation is memory usage. The program reads the whole input text into memory and also builds the encoded bit string before writing it. This is fine for coursework samples but not ideal for very large files. A stronger version could process the file in blocks.

The project currently focuses on text files. Binary file compression would require careful reading and writing of bytes. That would make the program more general but also more complicated.

The program could also show the generated Huffman codes in a table. This would be useful for presentation because the audience could see which characters received short codes and which received long codes.

Conclusion

This coursework implemented a small Huffman file compression tool. The program can read a text file, count character frequencies, build a Huffman tree, create binary codes, compress the data, and decompress it back to the original text.

The aim of the project was achieved. The implementation uses important data structures from the Algorithms and Data Structures course: dictionaries, a min heap priority queue, and a binary tree. It also uses tree traversal and a greedy algorithm.

The tests showed that the decompressed files matched the original files. This means the compression is lossless. The repeated sample also showed that Huffman coding can reduce file size when the input has repeated symbols.

The project also showed an important practical point: compression is not always smaller for very small files. Because the program stores the frequency table in the compressed file, the header can make small compressed files larger. This is a useful result to mention because it shows the difference between algorithm theory and real file storage.

In the future, the project could be improved by using a smaller binary header, supporting binary files, processing large files in blocks, and displaying Huffman code tables for better explanation during the demo.

References

[1] Huffman, D. A. (1952). A Method for the Construction of Minimum-Redundancy Codes. Proceedings of the IRE, 40(9), 1098-1101.

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). Introduction to Algorithms. 3rd edition. MIT Press.

[3] Goodrich, M. T., Tamassia, R., and Goldwasser, M. H. (2014). Data Structures and Algorithms in Python. Wiley.

[4] Python Software Foundation. Python documentation for heapq, json, os, and sys modules. https://docs.python.org/3/

[5] GeeksforGeeks. Huffman Coding Greedy Algorithm. https://www.geeksforgeeks.org/huffman-coding-greedy-algo-3/

[6] Program source code written for this coursework project: main.py and huffman.py.

Appendix A. Sample input

The normal sample file contains this text:

```
Algorithms and data structures are important in computer science.
Huffman coding is used for lossless data compression.
Characters that appear many times get shorter binary codes.
Characters that appear only a few times get longer binary codes.
```

Appendix B. Main source code excerpt

The full source code is stored in the project folder. The most important part of the compression function is shown below.

```
def compress(src, dst):
with open(src, "r", encoding="utf-8") as f:
text = f.read()
```

```
freq = make_freq(text)
root = make_tree(freq)
codes = make_codes(root)
bits = ""
```

```
for ch in text:
bits += codes[ch]
```

```
data, pad = make_bytes(bits)
head = {"freq": freq, "pad": pad}
head_data = json.dumps(head, ensure_ascii=False).encode("utf-8")
```

```
with open(dst, "wb") as f:
f.write(len(head_data).to_bytes(4, "big"))
f.write(head_data)
f.write(data)
```

Appendix C. Demo plan

For the presentation, the project can be demonstrated in four short steps.

First, open the sample input file and show that it is normal readable text.

Second, run the compression command:

```
python main.py compress samples/repeated.txt samples/repeated.huff
```

Third, run the decompression command:

```
python main.py decompress samples/repeated.huff samples/repeated_result.txt
```

Fourth, compare the original and decompressed file:

```
diff samples/repeated.txt samples/repeated_result.txt
```

If there is no output from diff, it means the files are exactly the same. After that, the compression statistics can be explained. This is enough for a short demo because it shows both compression and correctness.